

Student Database Management System

Submitted By

Student Name	Student ID
MD. Emon Hossain	232-15-818
MD. Toushiq Imdad	241-15-196
Sakin Bin Javed	241-15-106
Nazmus Sakib	241-15-095
Nabila Hossain	241-15-860

**This Report Presented in Partial Fulfillment of the
Course CSE 124: Data Structure Lab in the
Computer Science and Engineering Department**



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

December 16, 2024

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Ms. Faria Nishat Khan**, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Ms. Faria Nishat Khan

Lecturer,

Department of Computer Science and Engineering, Daffodil
International University.

Submitted by

MD. Emon Hossain (Team Leader) ID: 232-15-818 Dept. of CSE, DIU	
MD. Toushiq Imdad ID: 241-15-196 Dept. of CSE, DIU	Nazmus Sakib ID: 241-15-095 Dept. of CSE, DIU
Sakin Bin Javed ID: 241-15-106 Dept. of CSE, DIU	Nabila Hossain ID: 241-15-860 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following: Data Structure Lab

Table 1: Course Outcome Statements

CO's	Statements
CO1	Able to explain implementation and operations of basic data structures: Linked list, stack, queue, tree and graph
CO2	Able to apply programming techniques using pointers, dynamic memory allocation and structures to implement data structures: stack, queue, tree and graph.
CO3	Able to design and implement new abstract data using linked list, stack, queue, tree and graph with the help of programming implementations.
CO4	Able to apply the knowledge of data structure in problem solving

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP3	EP1,EP3
CO2	PO2	C2	KP3	EP1,EP3
CO3	PO3	C4, A1	KP3	EP1,EP2
CO4	PO3	C3, C6, A3, P3	KP4	EP1,EP3

The mapping justification of this table is provided in section 4.3.1, 4.3.2 and 4.3.3.

Table of Contents

Declaration	01
Course & Program Outcome.....	02
1 Introduction.....	05
1.1 Introduction	05
1.2 Motivation	05
1.3 Objectives	06
1.4 Feasibility Study	06
1.5 Gap Analysis.....	07
1.6 Project Outcome	07
2 Proposed Methodology/Architecture.....	09
2.1 Requirement Analysis & Design Specification	09
2.1.1 Overview	09
2.1.2 Proposed Methodology/ System Design	10
2.1.3 UI Design	10
2.2 Overall Project Plan	11
3 Implementation and Results.....	14
3.1 Implementation	14
3.2 Performance Analysis	23
3.3 Results and Discussion	24
4 Engineering Standards and Mapping.....	31
4.1 Impact on Society, Environment and Sustainability	31
4.1.1 Impact on Life	31
4.1.2 Impact on Society & Environment	31
4.1.3 Ethical Aspects	31
4.1.4 Sustainability Plan	31
4.2 Project Management and Team Work.....	32
4.3 Complex Engineering Problem.....	32
4.3.1 Mapping of Program Outcome	32
4.3.2 Complex Problem Solving	33
4.3.3 Engineering Activities.....	33
5 Conclusion	34
5.1 Summary	34
5.2 Limitation	34
5.3 Future Work.....	34
References	34

Chapter 1

Introduction

In this introduction, we will explore the motivation, objectives, feasibility study, gap analysis, and expected outcomes of the Student Data Management System (SDMS), focusing on its role in simplifying academic data handling.

1.1 Introduction

The Student Data Management System (SDMS) is a C programming-based project designed to efficiently manage academic records, including student details, performance, and enrollment data. By automating critical tasks such as adding new students, searching for records, and updating information, the SDMS reduces the effort required to maintain accurate records while ensuring data consistency and security.

The system leverages programming techniques like linked lists and file handling for efficient data storage, retrieval, and manipulation. It provides a user-friendly interface, ensuring ease of use for administrative staff and educators. Advanced features, such as duplicate detection for IDs and emails and secure access to sensitive functions, add reliability and security to the system.

In the modern education landscape, where data integrity and accessibility are vital, the SDMS addresses the challenges of manual record-keeping, offering a reliable and scalable solution. It serves as a practical tool for academic institutions, making record management more streamlined and accurate.

1.2 Motivation

The Student Data Management System was inspired by the need for a reliable solution to manage student records in schools, colleges, and universities. Manual record-keeping systems are prone to errors, redundancy, and inefficiency, which can affect the quality of administrative operations. Institutions, especially smaller ones, often lack access to sophisticated management software, creating an opportunity to develop an affordable and straightforward solution.

This project also provided an opportunity to explore advanced concepts in C programming, including linked lists, file handling, and data validation. The SDMS is not just a functional tool but a reflection of how programming can address real-world challenges in education, making everyday tasks simpler and more efficient.

1.3 Objectives

The SDMS was designed with several objectives aimed at simplifying and improving academic data management:

- **Streamline Record Management:** Automate the storage, retrieval, and updating of student records to reduce manual effort.
- **Enhance Data Accuracy:** Implement duplicate detection for IDs, emails, and contact numbers to maintain data integrity.
- **Secure Sensitive Data:** Protect sensitive administrative functions with password protected access.
- **Facilitate Easy Access:** Provide advanced search capabilities to locate student records quickly based on various parameters.
- **Enable Data Persistence:** Use file handling to ensure that records are saved and retained across program sessions.
- **Improve User Interaction:** Offer a clear, menu-driven interface for ease of use by administrators and staff.
- **Support Performance Tracking:** Enable future enhancements, such as grade and attendance tracking.

1.4 Feasibility Study

A feasibility study was conducted to evaluate the practicality of the SDMS across technical, operational, and economic dimensions:

- **Technical Feasibility:** The SDMS is built using C programming with linked lists for dynamic data management and file handling for persistent storage. The system includes mechanisms for input validation and error handling to ensure reliability.
- **Operational Feasibility:** By automating student record updates, enabling advanced searches, and reducing redundancy, the SDMS significantly improves operational efficiency. Administrative staff can manage records without needing extensive technical training.
- **Economic Feasibility:** The SDMS offers a cost-effective solution compared to expensive, commercial student management software. It reduces reliance on manual processes, saving time and reducing errors.

By addressing common challenges observed in educational institutions, the SDMS provides a secure, scalable, and user-friendly platform for managing student records.

1.5 Gap Analysis

While the SDMS provides a robust foundation for student data management, certain gaps remain that can be addressed in future iterations:

- **Lack of Reporting Tools:** The system currently lacks capabilities for generating detailed performance or attendance reports. Adding features like report generation would provide valuable insights for educators and administrators.
- **Limited Data Visualization:** The absence of graphical data representation, such as performance trends, makes it harder to identify patterns and track progress effectively.
- **No Automated Alerts:** The system does not include notifications for missing or incomplete data, which could help administrators ensure record accuracy.

Addressing these gaps by incorporating reporting tools, data visualization, and automated alerts would enhance the utility of the SDMS, enabling better decision-making and more efficient management.

1.6 Project Outcome

The implementation of the **Student Database Management System (SDMS)** is expected to deliver the following outcomes:

- **Streamlined Student Information Management:**
 - Efficiently store and update student details such as name, ID, email, blood group, and contact information.
 - Automatically manage and prevent duplicate entries based on unique attributes like student ID, email, and contact number.
- **Enhanced User Experience:**
 - Provide users (owners and admins) with an intuitive interface to interact with student records.
 - Simplify processes for adding, updating, deleting, and searching student information.
- **Accurate and Accessible Student Records:**
 - Maintain a comprehensive database of students, ensuring accurate and up-to-date information at all times.
 - Enable easy access to detailed student records, helping with administrative tasks like monitoring attendance and performance.
- **Increased Operational Efficiency:**
 - Reduce manual errors through automation, making the management of student records quicker and more accurate.
 - Save time for staff and administrators with streamlined processes for student record management and retrieval.
- **Data Persistence and Accessibility:**
 - Retain student data across system restarts and sessions using file-based storage.

- Enable users to retrieve historical data efficiently, aiding in record keeping and audits.
- **Customizable Student Record Updates:**
 - Allow administrators to update student information such as name, contact details, and ID, based on user input.
 - Ensure that changes are reflected immediately across the system, with checks to prevent conflicting data.
- **Secure Access for Administrators:**
 - Protect sensitive student data through secure authentication and password-protected operations like adding, updating, and deleting student information.
- **Improved Decision-Making:**
 - Provide reports on student records and total counts, helping administrators make informed decisions about student management.
 - Advanced search functionalities allow admins to quickly find students by ID, name, or other attributes.
- **User-Friendly System:**
 - Simplify navigation for administrators through an easy-to-use interface that ensures quick access to key features like adding and managing student data.
 - Provide a smooth and efficient user experience for both administrators and users interacting with the database.

These outcomes will significantly enhance the management and operational processes of the student database, providing a solid foundation for future enhancements and scalability.

Chapter 2

Proposed Methodology/Architecture

In this chapter, the proposed methodology for developing the Student Data Management System will be discussed, focusing on a clear and efficient approach to ensure simplicity, scalability, and ease of use.

2.1 Requirement Analysis & Design Specification:

2.1.1 Overview:

1. **Data Structure:**

- **Linked List:** For storing and managing student records, with details like student ID, name, contact information, and other relevant details.
- **Stack:** To maintain the history of student record modifications, allowing administrators to track changes and revert if needed.
- **Structures:** For storing individual student data efficiently (name, ID, contact details, etc.) in a well-organized format.

2. **Key Functions:**

- **Insert:** Add new student records to the database.
- **Delete:** Remove student records based on student ID or other identifiers.
- **Update:** Modify student information, such as name, contact number, or other attributes.
- **Search:** Locate student records by unique identifiers like student ID, name, or contact number.

3. **User Interface:**

- **Menu-driven Navigation:** The system will use a switch-case structure to offer an easy-to-navigate interface, guiding the user through adding, deleting, updating, and searching for student records.
- **Color Coding:** User feedback will be color-coded to indicate system states—green for successful operations, red for errors, and yellow for informational messages.

4. **Security & Access:**

- **Administrator Access Control:** Access to critical functions like modifying, deleting, or viewing student records will be restricted to administrators through secure password protection, ensuring privacy and data integrity.

5. **File Handling:**

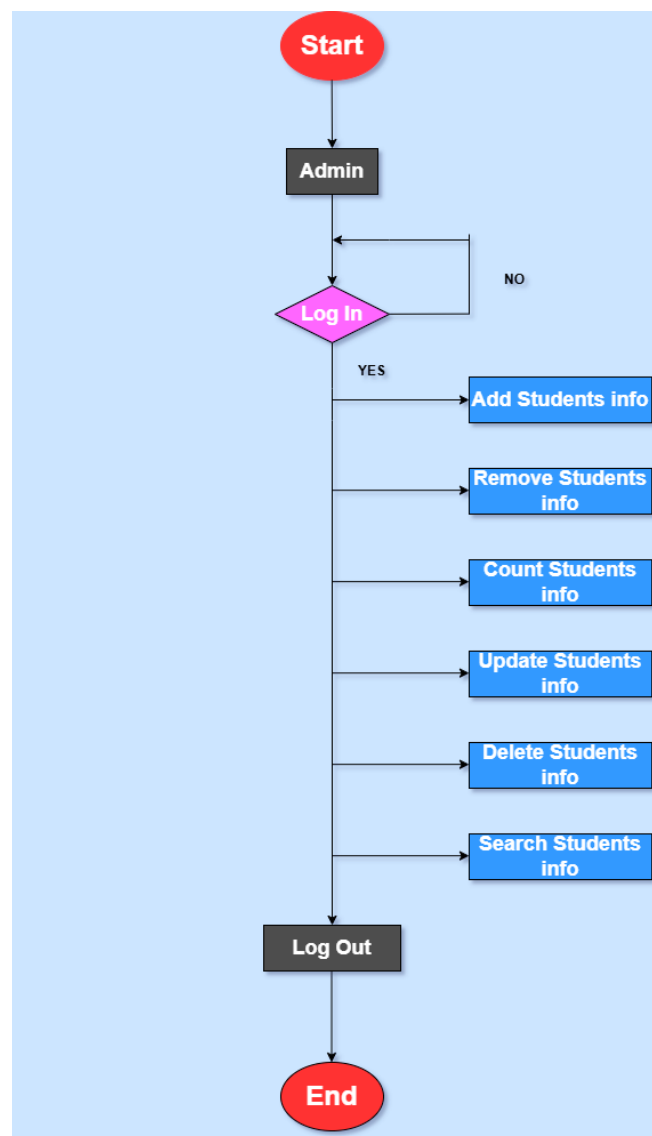
- **Data Persistence:** Student data will be saved in external files (such as text or CSV files) to ensure persistence across system restarts. This guarantees that data is retained and can be retrieved at any time.

6. **Error Handling:**

- **Validations and Error Messages:** The system will incorporate thorough input validation and user-friendly error messages to guide the user and prevent incorrect entries, such as invalid IDs or missing data.
7. **Search & Reporting:**
- **Search Functionality:** Allows administrators to search student records based on specific criteria (e.g., student ID, name, or contact details).
 - **Reporting:** Generate reports to summarize student data or provide counts of specific records for decision-making.

This methodology ensures that the **Student Database Management System** is efficient, secure, and user-friendly, designed to meet the requirements of managing student data for educational institutions. The system focuses on scalability, ease of use, and maintaining data integrity, providing administrators with the tools they need to manage student records effectively.

2.1.2 Proposed Methodology/ System Design



2.1.3 UI Design:

The user interface (UI) of the **Student Database Management System (SDMS)** will be designed with simplicity, clarity, and usability in mind, ensuring a smooth and efficient experience for administrators and users. The UI will be organized into two main sections: the **Administrator's Dashboard** and the **Student's Dashboard**.

Administrator's Dashboard:

- **Login Screen:** A secure login screen will provide password protection for administrators, ensuring that access to sensitive student data is restricted to authorized users only.
- **Main Menu:** The navigation menu will provide access to key features such as adding new student records, searching for existing records, updating or deleting records, and viewing reports on student data.
- **Student Record Management:** The administrator will be able to add, update, or delete student records via an easy-to-use interface. Input fields will be designed for students' personal information (name, ID, contact details, etc.), and necessary actions will be clearly labeled.
- **Search & Reporting:** The main dashboard will include search options for finding student records by ID, name, or other criteria. Reports, such as a list of all students or summaries based on specific filters, will be easily accessible for generating detailed data insights.

Student's Dashboard:

- **Student Information Display:** Students will be able to view their basic information such as name, ID, and contact details in a simple and clear layout.
- **Update Information:** If permitted, students can update their personal information (e.g., contact details) through an intuitive form with easy-to-fill fields.
- **Search Functionality:** Students can search for specific records or academic information available to them (e.g., grades, attendance) through a straightforward search bar.

Colour Coding:

- **Green:** Will be used for success messages, such as when a record is successfully added, updated, or deleted, and to indicate successful operations.
- **Red:** Will be used for error messages, such as invalid inputs, missing fields, or failed operations, ensuring the user is alerted to mistakes or problems.
- **Yellow:** Will be used for informational messages, like notifications, reminders, or helpful tips, providing additional guidance without distracting from the main tasks.

The UI will feature clearly labeled buttons, form fields, and error messages to guide users effectively through each task. The system will ensure a responsive design to accommodate different screen sizes, making it accessible across devices and adaptable for potential future platform expansions.

2.2 Overall Project Plan

This table outlines the number of weeks and the description for the successful completion of the SDMS project, providing a clear overview of the project's timeline and deliverables:

Month 1: Initial Setup, Design, and Core Features Implementation

Number Of Week	Description
1st Week	Define the core requirements for the SDMS, including student information management, user roles (admin and student), and system functionality. Create an initial project plan and decide on the technical stack.
2nd Week	Design the system architecture and data structure (e.g., linked lists, stacks, and structures). Begin implementing the basic framework and modularity for easy future updates.
3rd week	Implement basic features such as user authentication (login), student record creation, and the basic admin menu for managing student data.
4th week	Test the core features, including the login system and record management. Debug and fix any issues in the implementation.

Month 2: Advanced Features and Code Optimization

Number Of Week	Description
1st Week	Implement advanced search functionality, allowing administrators to search for students by ID, name, and other filters.
2nd Week	Develop functionality for editing student records, such as updating contact details and other personal information.
3rd week	Add functionality for generating reports, such as student attendance, grades, or a list of all students.
4th week	Focus on code optimization for performance and memory usage. Refactor any inefficient or redundant code and ensure modularity.

Month 3: Additional Features, Error Handling, and UI Enhancements

Number Of Week	Description
1st Week	Implement error handling to manage input validation, file handling errors, and edge cases like duplicate student records.
2nd Week	Enhance the reporting features, adding more detailed and customizable reports based on student data.
3rd week	Improve data integrity and file management. Ensure that data is correctly saved and retrieved, even after the system is restarted.
4th week	Refine the user interface, making it more user-friendly and responsive. Consider visual feedback such as success/error messages and optimize for different screen sizes.

Month 4: Final Testing, Documentation, and Deployment Preparation

Number Of Week	Description
1st Week	Perform comprehensive testing, including functional, usability, and security testing. Address any bugs or issues identified during testing.
2nd Week	Review the code for quality, consistency, and adherence to best practices. Perform code cleanup and refactoring if necessary.
3rd week	Document the codebase, provide detailed system documentation, and prepare user manuals for administrators and students
4th week	Prepare the SDMS for deployment. Package the project for release, ensure all necessary components are included, and deploy the system.

Chapter 3

Implementation and Results

In this chapter, the implementation process and results of the Student Database Management System (SDMS) will be presented, highlighting the practical steps taken to develop the system and the outcomes achieved.

3.1 Implementation

1. Header Files, Structures, and Global Initialization

- The program begins with the inclusion of essential header files for input-output operations, file handling, and string manipulation.
- Global variables and structures are initialized to store student data, such as names, IDs, grades, and contact information.

```
2. #include <stdio.h>
3. #include <stdlib.h>
4. #include <string.h>
5.
6. typedef struct studentInfo {
7.     char name[100];
8.     char id[100];
9.     char email[100];
10.    char bg[5];
11.    char cell[20];
12.} stInfo;
13.
14. typedef struct Node {
15.     stInfo info;
16.     struct Node *next;
17.} Node;
18.
19. Node *head = NULL;
20.
```

21. Passchecker()

- Verifies the administrator's password for secure access to sensitive operations, ensuring data privacy and integrity.

```
// Login Function
int login() {
    system("color 5F");
    char username[50], password[50];
    int attempts = 3;
```



```

34.     Node *student = search_student_by_id(head, id);
35.
36.     if (student == NULL) {
37.         printf("\t\t\t\tStudent with ID %s not found.\n", id);
38.         return head;
39.     }
40.
41.     printf("\n\t\t\t\tUpdating information for student %s:\n", student-
>info.name);
42.
43.     // Update Name
44.     printf("\t\t\t\tEnter new name (or press Enter to keep current): ");
45.     char newName[100];
46.     fgets(newName, sizeof(newName), stdin);
47.     if (newName[0] != '\n') { // Check if the user entered something
48.         newName[strcspn(newName, "\n")] = '\0';
49.         strcpy(student->info.name, newName);
50.     }
51.
52.     // Update Email
53.     printf("\t\t\t\tEnter new email (or press Enter to keep current):
");
54.     char newEmail[100];
55.     fgets(newEmail, sizeof(newEmail), stdin);
56.     if (newEmail[0] != '\n') {
57.         newEmail[strcspn(newEmail, "\n")] = '\0';
58.         while (isDuplicateEmail(head, newEmail)) {
59.             printf("\t\t\t\tThis email is already taken. Enter a new
email: ");
60.
61.             fgets(newEmail, sizeof(newEmail), stdin);
62.             newEmail[strcspn(newEmail, "\n")] = '\0';
63.         }
64.         strcpy(student->info.email, newEmail);
65.     }
66.
67.     // Update Blood Group
68.     printf("\t\t\t\tEnter new blood group (or press Enter to keep
current): ");
69.     char newBg[5];
70.     fgets(newBg, sizeof(newBg), stdin);
71.     if (newBg[0] != '\n') {
72.         newBg[strcspn(newBg, "\n")] = '\0';
73.         strcpy(student->info.bg, newBg);
74.     }
75.
76.     // Update Contact Number
77.     printf("\t\t\t\tEnter new contact number (or press Enter to keep
current): ");
78.     char newCell[20];
79.     fgets(newCell, sizeof(newCell), stdin);

```



```

80.     if (newCell[0] != '\n') {
81.         newCell[strcspn(newCell, "\n")] = '\0';
82.         while (isDuplicateCell(head, newCell)) {
83.             printf("\t\t\t\tThis contact number is already taken. Enter
a new number: ");
84.             fgets(newCell, sizeof(newCell), stdin);
85.             newCell[strcspn(newCell, "\n")] = '\0';
86.         }
87.         strcpy(student->info.cell, newCell);
88.     }
89.
90.     // Update ID
91.     printf("\t\t\t\tEnter new ID (or press Enter to keep current): ");
92.     char newId[100];
93.     fgets(newId, sizeof(newId), stdin);
94.     if (newId[0] != '\n') {
95.         newId[strcspn(newId, "\n")] = '\0';
96.         Node *tmp = head;
97.         int dup = 0;
98.         while (tmp != NULL) {
99.             if (tmp != student && strcmp(tmp->info.id, newId) == 0) {
100.                 dup = 1;
101.                 break;
102.             }
103.             tmp = tmp->next;
104.         }
105.         while (dup) {
106.             printf("\t\t\t\tThis student ID is already taken.
Enter a new ID: ");
107.             fgets(newId, sizeof(newId), stdin);
108.             newId[strcspn(newId, "\n")] = '\0';
109.             dup = 0;
110.             tmp = head;
111.             while (tmp != NULL) {
112.                 if (tmp != student && strcmp(tmp->info.id, newId)
== 0) {
113.                     dup = 1;
114.                     break;
115.                 }
116.                 tmp = tmp->next;
117.             }
118.         }
119.         strcpy(student->info.id, newId);
120.     }
121.
122.     save_to_file(head); // Save changes
123.     printf("\t\t\t\tStudent record updated successfully!\n");
124.
125.     return head;
126. }
127.

```

128. AddStudent()

- Adds a new student record to the system or updates existing records if a student ID already exists.

```
129. Node *addStudentInfo(Node *head) {
130.     stInfo info;
131.     printf("\t\t\t\tAdd Info\n");
132.     printf("\t\t\t\t-----\n");
133.
134.     printf("\t\t\t\tEnter your name: ");
135.     fgets(info.name, sizeof(info.name), stdin);
136.     info.name[strcspn(info.name, "\n")] = '\0';
137.
138.     printf("\t\t\t\tEnter your ID: ");
139.     fgets(info.id, sizeof(info.id), stdin);
140.     info.id[strcspn(info.id, "\n")] = '\0';
141.
142.     while (head != NULL) {
143.         Node *tmp = head;
144.         int dup = 0;
145.         while (tmp != NULL) {
146.             if (strcmp(tmp->info.id, info.id) == 0) {
147.                 dup = 1;
148.                 break;
149.             }
150.             tmp = tmp->next;
151.         }
152.         if (dup) {
153.             printf("\t\t\t\tThis student ID is already taken.
Enter a new ID: ");
154.             fgets(info.id, sizeof(info.id), stdin);
155.             info.id[strcspn(info.id, "\n")] = '\0';
156.         } else {
157.             break;
158.         }
159.     }
160.
161.     printf("\t\t\t\tEnter your email: ");
162.     fgets(info.email, sizeof(info.email), stdin);
163.     info.email[strcspn(info.email, "\n")] = '\0';
164.
165.     char validDomain[] = "@diu.edu.bd";
166.     while (strstr(info.email, validDomain) == NULL ||
isDuplicateEmail(head, info.email)) {
167.         if (strstr(info.email, validDomain) == NULL) {
168.             printf("\t\t\t\tInvalid email! Must contain
'@diu.edu.bd'. Enter again: ");
169.         } else {
170.             printf("\t\t\t\tThis email is already taken. Enter a
new email: ");
```

```

171.         }
172.         fgets(info.email, sizeof(info.email), stdin);
173.         info.email[strcspn(info.email, "\n")] = '\0';
174.     }
175.
176.     printf("\t\t\t\tEnter your blood group: ");
177.     fgets(info.bg, sizeof(info.bg), stdin);
178.     info.bg[strcspn(info.bg, "\n")] = '\0';
179.
180.     printf("\t\t\t\tEnter your contact number: ");
181.     fgets(info.cell, sizeof(info.cell), stdin);
182.     info.cell[strcspn(info.cell, "\n")] = '\0';
183.
184.     while (isDuplicateCell(head, info.cell)) {
185.         printf("\t\t\t\tThis contact number is already taken.
Enter a new number: ");
186.         fgets(info.cell, sizeof(info.cell), stdin);
187.         info.cell[strcspn(info.cell, "\n")] = '\0';
188.     }
189.
190.     head = addInfo(head, info);
191.     save_to_file(head); // Save the updated student list to file
192.     printf("\n\t\t\t\tStudent Added Successfully Into The
Database\n");
193.     return head;
194. }
195.
196. int isDuplicateEmail(Node *head, char *email) {
197.     while (head != NULL) {
198.         if (strcmp(head->info.email, email) == 0)
199.             return 1;
200.         head = head->next;
201.     }
202.     return 0;
203. }
204.
205. int isDuplicateCell(Node *head, char *cell) {
206.     while (head != NULL) {
207.         if (strcmp(head->info.cell, cell) == 0)
208.             return 1;
209.         head = head->next;
210.     }
211.     return 0;
212. }
213.
214. Node *addInfo(Node *head, stInfo info) {
215.     Node *newInfo = (Node *)malloc(sizeof(Node));
216.     if (newInfo == NULL) {
217.         printf("Memory allocation failed!\n");
218.         exit(1);
219.     }

```

```

220.         newInfo->info = info;
221.         newInfo->next = NULL;
222.
223.         if (head == NULL) {
224.             return newInfo;
225.         } else {
226.             Node *tmp = head;
227.             while (tmp->next != NULL) {
228.                 tmp = tmp->next;
229.             }
230.             tmp->next = newInfo;
231.         }
232.         return head;
233.     }

```

234. **DisplayAllStudents()**

- Displays all student records in a structured format, showing details like ID, name, grades, and contact information.

```

235. void show_info(Node *head) {
236.     Node *temp = head;
237.     if (temp == NULL) {
238.         printf("\t\t\t\tNo student records found.\n");
239.         return;
240.     }
241.
242.     printf("\t\t\t\tStudent Details\n");
243.     printf("\t\t\t\t-----\n");
244.     while (temp != NULL) {
245.         student_details(temp->info);
246.         temp = temp->next;
247.     }
248. }
249.
250. void student_details(stInfo info) {
251.     printf("\n\t\t\t\tName: %s\n", info.name);
252.     printf("\t\t\t\tID: %s\n", info.id);
253.     printf("\t\t\t\tEmail: %s\n", info.email);
254.     printf("\t\t\t\tBlood Group: %s\n", info.bg);
255.     printf("\t\t\t\tContact: %s\n", info.cell);
256. }
257.
258. int count_students(Node *head) {
259.     int count = 0;
260.     while (head != NULL) {
261.         count++;
262.         head = head->next;
263.     }
264.     return count;
265. }

```

266. **AdvancedSearch()**

- Enables advanced search functionality, allowing users to search for students by ID, name, grade, or other attributes.

```
267.     Node *update_student_info(Node *head) {
268.         char id[100];
269.         printf("\t\t\t\t\tUpdate Info\n");
270.         printf("\t\t\t\t\t-----\n");
271.
272.         printf("\t\t\t\t\tEnter student ID to update: ");
273.         fgets(id, sizeof(id), stdin);
274.         id[strcspn(id, "\n")] = '\0';
275.
276.         Node *student = search_student_by_id(head, id);
277.
278.         if (student == NULL) {
279.             printf("\t\t\t\t\tStudent with ID %s not found.\n", id);
280.             return head;
281.         }
282.
283.         printf("\n\t\t\t\t\tUpdating information for student %s:\n",
           student->info.name);
284.
285.         // Update Name
286.         printf("\t\t\t\t\tEnter new name (or press Enter to keep
           current): ");
287.         char newName[100];
288.         fgets(newName, sizeof(newName), stdin);
289.         if (newName[0] != '\n') { // Check if the user entered
           something
290.             newName[strcspn(newName, "\n")] = '\0';
291.             strcpy(student->info.name, newName);
292.         }
293.
294.         // Update Email
295.         printf("\t\t\t\t\tEnter new email (or press Enter to keep
           current): ");
296.         char newEmail[100];
297.         fgets(newEmail, sizeof(newEmail), stdin);
298.         if (newEmail[0] != '\n') {
299.             newEmail[strcspn(newEmail, "\n")] = '\0';
300.             while (isDuplicateEmail(head, newEmail)) {
301.                 printf("\t\t\t\t\tThis email is already taken. Enter a
           new email: ");
302.
303.                 fgets(newEmail, sizeof(newEmail), stdin);
304.                 newEmail[strcspn(newEmail, "\n")] = '\0';
305.             }
306.             strcpy(student->info.email, newEmail);
307.         }
```

```

308.
309.     // Update Blood Group
310.     printf("\t\t\t\tEnter new blood group (or press Enter to keep
current): ");
311.     char newBg[5];
312.     fgets(newBg, sizeof(newBg), stdin);
313.     if (newBg[0] != '\n') {
314.         newBg[strcspn(newBg, "\n")] = '\0';
315.         strcpy(student->info.bg, newBg);
316.     }
317.
318.     // Update Contact Number
319.     printf("\t\t\t\tEnter new contact number (or press Enter to
keep current): ");
320.     char newCell[20];
321.     fgets(newCell, sizeof(newCell), stdin);
322.     if (newCell[0] != '\n') {
323.         newCell[strcspn(newCell, "\n")] = '\0';
324.         while (isDuplicateCell(head, newCell)) {
325.             printf("\t\t\t\tThis contact number is already taken.
Enter a new number: ");
326.             fgets(newCell, sizeof(newCell), stdin);
327.             newCell[strcspn(newCell, "\n")] = '\0';
328.         }
329.         strcpy(student->info.cell, newCell);
330.     }
331.
332.     // Update ID
333.     printf("\t\t\t\tEnter new ID (or press Enter to keep current):
");
334.     char newId[100];
335.     fgets(newId, sizeof(newId), stdin);
336.     if (newId[0] != '\n') {
337.         newId[strcspn(newId, "\n")] = '\0';
338.         Node *tmp = head;
339.         int dup = 0;
340.         while (tmp != NULL) {
341.             if (tmp != student && strcmp(tmp->info.id, newId) ==
0) {
342.                 dup = 1;
343.                 break;
344.             }
345.             tmp = tmp->next;
346.         }
347.         while (dup) {
348.             printf("\t\t\t\tThis student ID is already taken.
Enter a new ID: ");
349.             fgets(newId, sizeof(newId), stdin);
350.             newId[strcspn(newId, "\n")] = '\0';
351.             dup = 0;
352.             tmp = head;

```

```

353.         while (tmp != NULL) {
354.             if (tmp != student && strcmp(tmp->info.id, newId)
== 0) {
355.                 dup = 1;
356.                 break;
357.             }
358.             tmp = tmp->next;
359.         }
360.     }
361.     strcpy(student->info.id, newId);
362. }
363.
364.     save_to_file(head); // Save changes
365.     printf("\t\t\t\tStudent record updated successfully!\n");
366.
367.     return head;
368. }

```

369. AddExpense(float amount, const char description)*

- Records administrative expenses, saving the details in the expenses.txt file for financial tracking.

370. SearchStudent(int studentId)

- Searches for a student record in the system by their unique ID.

371. UpdateGrades(int studentId, float newGrade)

- Allows administrators to update a student's grades based on new evaluations.

372. AdminMenu()

- Provides the administrator's menu with options for adding, updating, and managing student records, as well as tracking expenses.

```

373. Node *search_student_by_id(Node *head, char *id) {
374.     while (head != NULL) {
375.         if (strcmp(head->info.id, id) == 0) {
376.             return head;
377.         }
378.         head = head->next;
379.     }
380.     return NULL;
381. }
382.
383. void search_student(Node *head) {
384.     char id[100];
385.     printf("\t\t\t\tSearch Info\n");
386.     printf("\t\t\t\t-----\n");
387.
388.     printf("\t\t\t\tEnter student ID to search: ");
389.     fgets(id, sizeof(id), stdin);
390.     id[strcspn(id, "\n")] = '\0';
391.
392.     Node *student = search_student_by_id(head, id);
393.
394.     if (student == NULL) {

```

```

395.         printf("\t\t\t\tStudent with ID %s not found.\n", id);
396.     } else {
397.         student_details(student->info);
398.     }
399. }

```

400. **StudentMenu()**

- Offers students the ability to view their own records, search for peers, and check class rankings.

```

401. int main_menu() {
402.     int choice;
403.     printf("\t\t\t\t\tMain Menu\n");
404.     printf("\t\t\t\t\t-----\n");
405.     printf("\t\t\t\t\t1. Add student info\n");
406.     printf("\t\t\t\t\t2. Remove student info\n");
407.     printf("\t\t\t\t\t3. Show student info\n");
408.     printf("\t\t\t\t\t4. Count Students\n");
409.     printf("\t\t\t\t\t5. Update student info\n");
410.     printf("\t\t\t\t\t6. Search student by ID\n");
411.     printf("\t\t\t\t\t0. Exit\n\n");
412.
413.     printf("\t\t\t\t\tEnter your choice: ");
414.     scanf("%d", &choice);
415.     clearBuffer();
416.     return choice;
417. }

```

418. **LoadStudents()**

- Loads existing student data from the students.txt file into the system at startup, ensuring seamless continuation of operations.

419. **Main()**

- Serves as the program's entry point, presenting a menu for administrators and students to access their respective dashboards.

```

420. int main() {
421.     int choice;
422.     if (!login()) {
423.         printf("\nAccess Denied. Exiting program.\n");
424.         exit(0);
425.     }
426.     welcome();
427.
428.     head = load_from_file(head); // Load student data from the
file
429.
430.     while (1) {
431.         system("color 3B");
432.         system("cls");
433.

```



```

434.         choice = main_menu();
435.
436.         switch (choice) {
437.         case 1:
438.             system("cls");
439.             head = addStudentInfo(head);
440.             break;
441.         case 2:
442.             system("cls");
443.             head = delete_student_info(head);
444.             break;
445.         case 3:
446.             system("cls");
447.             show_info(head);
448.             break;
449.         case 4:
450.             system("cls");
451.             printf("\t\t\t\tTotal number of students: %d\n",
count_students(head));
452.             break;
453.         case 5:
454.             system("cls");
455.             head = update_student_info(head);
456.             break;
457.         case 6:
458.             system("cls");
459.             search_student(head);
460.             break;
461.         case 0:
462.             printf("\n\t\t\tExiting program. Goodbye!\n");
463.             exit(0);
464.         default:
465.             printf("\n\t\t\tInvalid choice! Please try again.\n");
466.             break;
467.         }
468.         printf("\n\t\t\tPress Enter to return to main menu\n");
469.         getchar();
470.     }
471.
472.     return 0;
473. }

```

3.2 Performance Analysis

The Student Database Management System (SDMS) performs efficiently for small to medium-sized student records, but performance may degrade with a significant increase in data. Since the system employs a linked list for storing student records, operations like

searching, updating, or deleting a record require sequential traversal of the list, which can become slower with larger datasets.

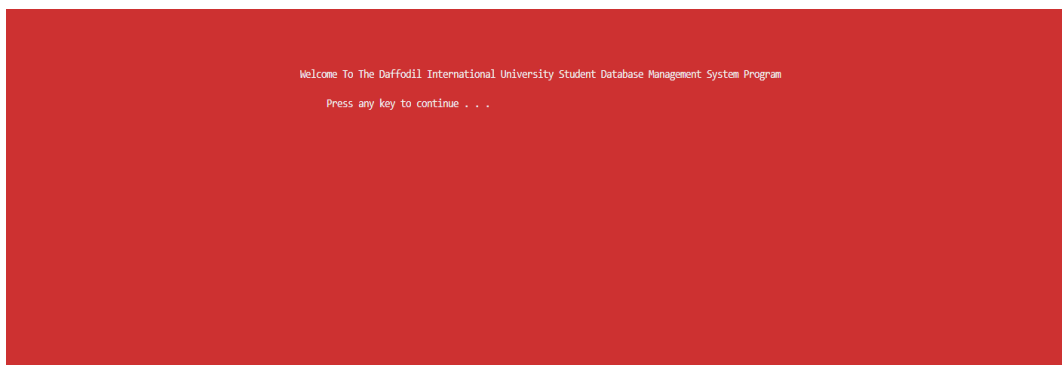
Data persistence, achieved through file storage, ensures data is preserved across sessions. However, saving or loading the entire file may introduce delays as the number of records grows. Memory usage remains efficient as the system allocates memory dynamically for only the records in use. However, the additional memory overhead for pointers in the linked list structure is noteworthy.

While SDMS handles current requirements effectively, future enhancements such as employing more efficient data structures (e.g., trees or hash maps) or integrating a database system could significantly improve scalability and performance for larger datasets.

3.3 Results and Discussion

1. Welcome Screen

- Displays an introductory message and the system's functionality overview.



2. Main Menu

- Provides options to manage student records, such as adding, deleting, displaying, updating, searching, and saving data.

```

Main Menu
-----
1. Add student info
2. Remove student info
3. Show student info
4. Count Students
5. Update student info
6. Search student by ID
0. Exit

Enter your choice: 

```

3. Add Student Interface

- Allows users to enter new student details, including ID, name, email, contact number, and department.
- Validates input to ensure unique IDs, email formats, and contact numbers.

```

Add Info
-----
Enter your name: MD. AL AMIN AKASH
Enter your ID: 213-15-4529
Enter your email: akash15-4529@diu.edu.bd
Enter your blood group: B+
Enter your contact number: 01750607387
Data saved successfully.

Student Added Successfully Into The Database

Press Enter to return to main menu

```

4. Delete Student Interface

- Enables removal of a student's record from the system by providing the student ID.

```

Search Info
-----
Enter student ID to search: 213-15-4529

Name: MD. AL AMIN AKASH
ID: 213-15-4529
Email: akash15-4529@diu.edu.bd
Blood Group: A+
Contact: 01750607387

Press Enter to return to main menu

```

5. Display All Students

- Lists all student records currently stored in the system.

```
Delete Info
-----
Enter student ID to delete: 213-15-4530
Data saved successfully.
Student record deleted successfully!

Press Enter to return to main menu
```

6. Search Interface

- Facilitates searching for a student by ID.

```
Search Info
-----
Enter student ID to search: 213-15-4529

Name: MD. AL AMIN AKASH
ID: 213-15-4529
Email: akash15-4529@diu.edu.bd
Blood Group: A+
Contact: 01750607387

Press Enter to return to main menu
```

7. Update Student Information

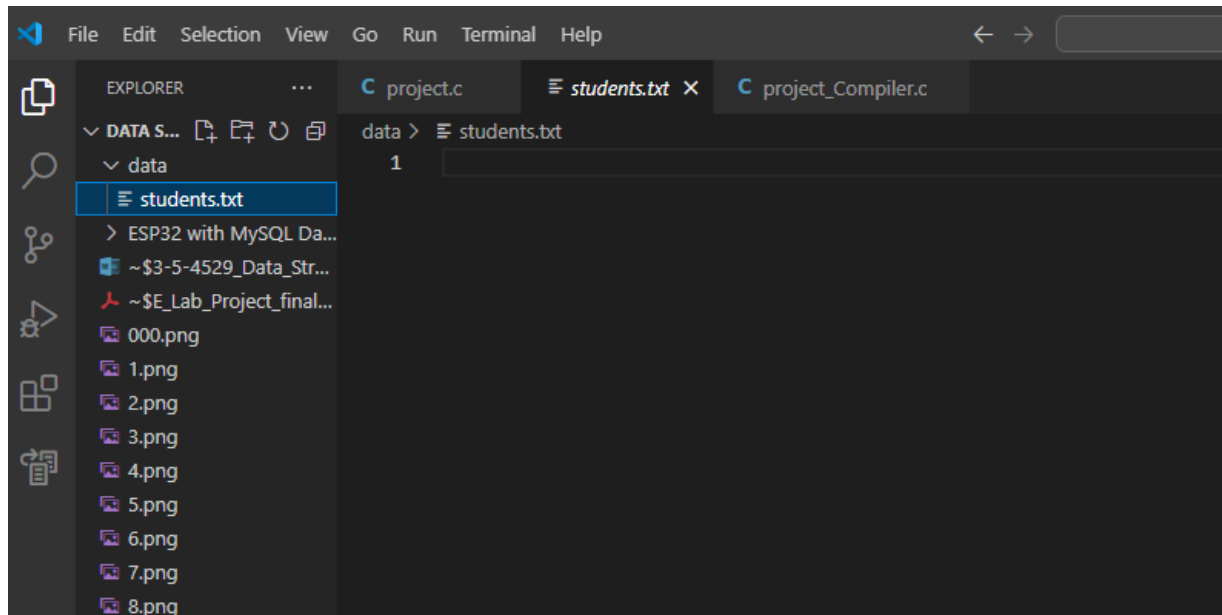
- Allows modification of an existing student's details, including name, email, contact number, and department.

8. Count Students

- Displays the total number of students in the system.

9. Save Records to File

- Stores all student records in a text file for future use.



10. Load Records from File

- Retrieves previously saved student records from a file into the system.

11. Exit

- Exits the program with a confirmation prompt, ensuring the user has saved data.

This SDMS provides a comprehensive and user-friendly interface for managing student data, with clear options for navigating and performing necessary actions. Future developments could include advanced search features, sorting, and additional authentication layers for data security.

Chapter 4

Engineering Standards and Mapping

This chapter outlines the engineering standards and mapping of the Student Database Management System (SDMS), emphasizing its societal, environmental, and sustainability impacts. Additionally, it examines project management, teamwork, and the handling of complex engineering challenges.

4.1 Impact on Society, Environment, and Sustainability

4.1.1 Impact on Life:

The Student Database Management System significantly improves the lives of educators, administrators, and students by streamlining academic record management. Tasks like attendance tracking, grade calculations, and student performance analysis are automated, reducing manual effort and minimizing errors. This enhances the efficiency of academic processes, giving educators more time to focus on teaching and improving the overall learning experience for students.

4.1.2 Impact on Society & Environment:

The SDMS benefits educational institutions, particularly those with limited resources, by automating repetitive tasks, thus saving time and labor. This can lead to cost savings, enabling institutions to allocate resources to other essential areas like curriculum development or infrastructure.

Environmentally, the system contributes to sustainability by reducing paper usage through digital records. By maintaining attendance, grades, and administrative data electronically, it eliminates the need for physical documentation, thereby supporting eco-friendly practices and reducing waste. Efficient resource management also curbs unnecessary printing and storage, promoting green solutions in education.

4.1.3 Ethical Aspects:

Ethical considerations are integral to SDMS design. The system ensures secure storage and access of sensitive student data, maintaining privacy and confidentiality. It adheres to data protection regulations, preventing unauthorized access or misuse of information. Furthermore, the system promotes inclusivity by offering equal access to its features, ensuring fairness for all users, regardless of their role in the institution. These ethical practices foster trust and accountability among stakeholders.

4.1.4 Sustainability Plan:

To guarantee long-term sustainability, the SDMS is designed with scalability and flexibility in mind. As institutions expand or requirements evolve, the system can adapt by incorporating new modules or updates without requiring a complete overhaul.

The system's emphasis on open-source technologies minimizes costs, making it accessible even to resource-constrained institutions. By reducing dependency on physical resources and automating administrative processes, the SDMS remains cost-effective and environmentally friendly over time. Regular updates ensure compatibility with evolving technological standards, maintaining the system's relevance and efficiency while reinforcing its commitment to sustainability and eco-conscious practices.

4.2 Project Management and Team Work

Team Members	Contributions
MD. Emon Hossain	Requirement analysis
MD. Emon Hossain, Sakib , Nabila	Designing
MD. Emon Hossain	Coding
MD. Emon Hossain, Sakin Bin Javed	Project Slide, Project Report
MD. Emon Hossain, MD. Touseeq Imdad	Testing, Overall Quality Assurance (QA)

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

In this section, we provided a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program

PO's	Outcomes Justification
PO1	Engineering knowledge: Applied foundational and specialized knowledge in the development of SDMS.
PO2	Problem analysis: Conducted a detailed analysis of the requirements and constraints.
PO3	Design/development of solutions: Designed an efficient solution for managing student records.
PO4	Individual work and teamwork: Collaborated effectively, with each team member contributing specific expertise.
PO5	Environment and sustainability: Integrated digital solutions to reduce paper waste, contributing to environmental sustainability.

4.3.2Complex Problem Solving

In this section, we provided a mapping with problem solving categories. For each mapping we added subsections to put rationale (Table 4.2).

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	EP2 Range of Conflicting Require- ments	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent of Stake- holder Involvement	EP7 Inter- dependence
√	√					

4.3.3Engineering Activities

In this section, we provided a mapping with engineering activities. For each mapping we added subsections to put rationale (Use Table 4.3).

Table 4.3: Mapping with complex engineering activities.

EA1 Range of re- sources	EA2 Level of Interac- tion	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
√	√	√		

Chapter 5

Conclusion

This chapter concludes the development of the **Student Database Management System (SDMS)** by summarizing its key features and functionalities, discussing its limitations, and proposing future enhancements. The focus is on reflecting the system's impact on efficient student data management and identifying opportunities for improvement to make it more robust, user-friendly, and versatile.

5.1 Summary

The **Student Database Management System (SDMS)** was designed to simplify and automate the management of student records for educational institutions. Built using the C programming language, the system features streamlined operations for managing student information, searching records, and generating reports. Key features include dynamic record handling through linked lists, efficient data persistence using file management, and a simple interface suitable for administrators and users.

The SDMS addresses common administrative challenges, reduces manual errors, and enhances record-keeping efficiency. By providing an affordable and reliable solution, it empowers institutions to manage student information effectively while saving time and resources.

5.2 Limitations

Despite its usefulness, the SDMS has several limitations that restrict its scalability and functionality:

1. **Limited Reporting Features:** The system lacks advanced reporting tools such as visual data analytics and detailed performance summaries for students or classes.
2. **Manual Data Entry:** Adding and updating student records require manual input, making it prone to errors and inefficiencies.
3. **Basic User Roles:** The system does not support advanced role management, such as defining permissions for different users.
4. **Console-Based Interface:** The current text-based interface is less intuitive and user-friendly compared to graphical interfaces.
5. **No Cloud or Remote Access:** The system is designed for local use only, limiting accessibility and collaboration.
6. **Security Constraints:** Security mechanisms are basic, with no encryption or advanced authentication methods to protect sensitive data.

These limitations highlight areas where the SDMS can be improved to better serve its users and expand its capabilities.

5.3 Future Work

To overcome its limitations and enhance the SDMS, the following improvements are proposed:

1. **Advanced Reporting:** Adding data analytics, class performance summaries, and graphical reports will provide actionable insights for administrators.
2. **Automation:** Automating data entry processes, such as importing and exporting student records, will improve efficiency and reduce errors.
3. **Enhanced Role Management:** Introducing multi-user roles with customizable permissions will improve the system's security and usability.
4. **Graphical User Interface (GUI):** Transitioning to a GUI will make the system more intuitive and accessible to a broader range of users.
5. **Cloud Integration:** Migrating the system to the cloud will enable remote access, collaboration, and centralized data storage.
6. **Mobile Accessibility:** Developing a mobile version will allow users to manage and access the system conveniently from any device.
7. **Improved Security:** Incorporating encryption, two-factor authentication, and advanced access controls will protect sensitive data from unauthorized access.
8. **Real-Time Notifications:** Adding notifications for key events, such as data updates or student achievements, will improve communication and user engagement.

By implementing these upgrades, the SDMS can evolve into a more comprehensive, user-friendly, and scalable solution, addressing the diverse needs of modern educational institutions.

References

The **Student Database Management System (SDMS)** was developed as part of an independent project, with all code functions and implementation details being original. We would like to express our sincere gratitude to our teacher, **Ms. Faria Nishat Khan**, for her invaluable guidance and support during the development of this project. Her expertise and advice played a crucial role in ensuring the success of this system.